

patch-hub: A Terminal-Based Tool to Streamline Linux Kernel Patch Review

Lorenzo Bertin Salvador, David Tadokoro, Hannah Harrisonn, and Paulo Meirelles

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Open Journals](#) ↗

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright¹⁴ and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))⁶.

Summary

Patch-hub is a terminal-based software written in Rust that aims to streamline one of the key workflows in the Linux kernel development model: reviewing patches. Its main features include browsing the patches of each Linux development mailing list, applying them locally for validation, and also the option to respond to them with a *Reviewed-by* tag.

Beyond its practical value, patch-hub is part of a broader effort to modernize Linux kernel development workflows and mitigate bottlenecks. By simplifying how reviewers and developers interact with patches, the tool not only reduces friction in the review process but also creates opportunities for empirical research on software engineering practices within this ecosystem.

Statement of need

A recent area of interest within the Linux kernel community is the long-term sustainability of its development process ([Tadokoro et al., 2025a](#)). For instance, there is concern that the workload of maintainers is not scaling and presents many challenges, as indicated by many community publications ([Corbet, 2013, 2018, 2021](#); [Dean, 2020](#); [Edge, 2013, 2016, 2018](#); [Vetter, 2017](#)); in particular, there is a mismatch between the volume of submitted patches and the capacity to review and integrate them into the project ([Rigby et al., 2014](#)). This scenario highlights potential bottlenecks in the workflow of maintainers, which may impact the project's growth.

On a broader scope, the Kworkflow (kw) project ([Tadokoro et al., 2025b](#)) is a hub of tools that aim to support the most varied workflows of kernel developers. In this sense, patch-hub is a sub-project of kw, with its dedicated codebase and focus on maintainers by directly addressing the bottlenecks associated with patch review. The tool enhances this workflow, which has traditionally involved fragmented, complex, and poorly standardized tasks. Furthermore, patch-hub can also serve as a central platform for research on the review cycle, enabling both a deeper understanding of the relationships among the actors involved and empirical evaluations of strategies to optimize this part of the development flow.

Introduction

The development of the Linux kernel is one of the most prominent examples of a large-scale Free Software project. Dozens of subsystems and thousands of contributors have allowed Linux to continue evolving for decades, making it the foundation of most modern computing systems. The project follows a rigorous review and integration process before a contribution, also known as a *patch*, can reach the software's end users.

In general, kernel development involves many repetitive tasks, both for contributors who seek to have their code incorporated and for maintainers who must ensure the high quality of the

39 contribution. Among these tasks, we emphasize compiling, running, and testing the Linux
40 kernel, as well as organizing, sending, and responding to patches. In practice, this translates
41 to executing long sequences of verbose commands, which waste considerable time to type, are
42 incredibly error-prone, and must be repeated multiple times throughout the development and
43 review of contributions.

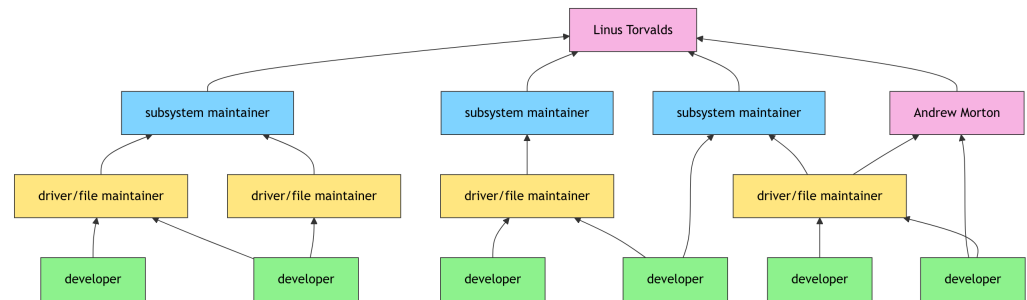


Figure 1: Lifecycle of a Linux patch from sending to merging into the upstream.

44 Due to the repetitive nature of these tasks, it is common for kernel developers to create or
45 adopt *ad hoc* scripts to automate such processes, in order to speed up execution and reduce
46 the likelihood of errors. As a result, this tooling is generally decentralized, leading to duplicated
47 efforts and contributing to the lack of robust standardized solutions for some of these tasks.

48 A Free Software project that aims to mitigate this issue is *kw*. Written in Bash, the software
49 helps Linux kernel developers perform various tasks through a unified Command-Line Interface
50 (CLI). Among many other features, users can seamlessly compile and deploy the kernel from
51 source, as well as manage multiple custom configurations for different environments and use
52 cases. In this way, *kw* directly addresses the main bottlenecks arising from repetitive tasks,
53 allowing developers to focus on reviewing and contributing patches themselves.

54 Within *kw*, another notable functionality, which has become an independent utility and is
55 the central topic of this paper, is *patch-hub*. With its dedicated repository and implemented
56 in Rust, *patch-hub* is a Terminal User Interface (TUI) focused on the interaction between
57 developers/maintainers and the *patchsets* (groups of related patches representing a single
58 contribution) representing the development of Linux. Each command in *kw* targets one or more
59 specific tasks, and in the case of *patch-hub*, its goal is to simplify user interaction with mailing
60 lists and the patchsets of each subsystem. Its main features include browsing mailing lists,
61 viewing all patchsets within a list, and interacting with individual patchsets, such as applying
62 one to the local kernel tree or saving a patchset for later analysis.

63 Under the hood, *patch-hub* leverages *Lore* (lore.kernel.org), the public archive of the Linux
64 development mailing lists, which supports searching for messages and patchsets on demand,
65 in contrast to the traditional model based on subscribing to the lists. Beyond its practical
66 value, *patch-hub* enables empirical investigations into the workflows of maintainers. With
67 appropriate data collection, it could even support studies in software engineering aimed at
68 maintaining large-scale projects.

69 patch-hub

70 Features

71 In general, the main features of *patch-hub* align with the goal presented in the previous
72 section: to simplify the interaction between those involved in kernel development (specifically
73 maintainers/reviewers) with the patchsets that represent this development.

74 It is worth noting that the project remains in continuous development, and some upcoming
75 features will be exposed in the **Next Steps** section. The following subsections present the most
76 relevant features currently implemented and available to the tool's end users.

77 **Integration with Linux development mailing lists**

```
patch-hub

linux-8086 - Linux-8086 Development Archive on lore.kernel.org
          linux-acpi - Linux ACPI
linux-admin - Linux-admin Development Archive on lore.kernel.org
          linux-alpha - Alpha arch development list
linux-amlogic - Linux-Amlogic Archive on lore.kernel.org
          linux-api - Linux userland API discussions
linux-arch - Generic Linux architectural discussions
linux-arm-kernel - Linux-ARM-Kernel Archive on lore.kernel.org
          linux-arm-msm - Linux ARM-MSM sub-architecture
linux-aspeed - Linux-Aspeed Archive on lore.kernel.org
          linux-assembly - Linux assembly list
linux-audit - Linux-audit Archive on lore.kernel.org
          linux-bcache - Linux bcache driver list
          linux-bcachefs - Linux bcachefs list
          linux-block - Linux block layer
> linux-bluetooth - Linux bluetooth development
          linux-btrace - Linux btrace development
          linux-btrfs - Linux Btrfs filesystem development
          linux-bugs - Generic list for bug discussions
linux-c-programming - Linux-c-programming Development Archive on lore.kernel.org
          linux-can - Linux CAN drivers development

Target List: linux- (ESC) to quit | (ENTER) to confirm | (?) help
```

Figure 2: Mailing list selection screen.

78 Users can browse the mailing lists of each subsystem available on lore.kernel.org. For each
79 list, users can navigate through the submitted patches, clustered in their respective patchsets,
80 from most recent to oldest, and analyze each one individually.

```
patch-hub

000. V01 | #31 | gpu: nova-core: firmware: Hopper/Blackwell support
001. V01 | #01 | rust: debugfs: Use kernel Atomic type in docs example
002. V06 | #08 | Rust bindings for gem shmem + iosys_map
003. V01 | #01 | rust: pci: fix build failure when CONFIG_PCI_MSI is disabled
004. V01 | #04 | Inline helpers into Rust without full LTO
005. V01 | #46 | Allow inlining C helpers into Rust when using LTO
006. V15 | #16 | Refcounted interrupts, SpinLockIrq for rust
007. V02 | #01 | rust: pwm: Add UnregisteredChip wrapper around Chip
008. V01 | #01 | rust_binder: remove spin_lock() in rust_shrink_free_page()
009. V06 | #01 | rust: Return Option from page_align and ensure no usize overflow
010. V02 | #01 | rust: num: bounded: mark __new as unsafe
> 011. V02 | #13 | gpu: nova-core: add Turing support
012. V03 | #01 | scripts: generate_rust_analyzer: Add message to sysroot assertion
013. V02 | #01 | rust: rbtree: fix minor typos in comments
014. V01 | #01 | rust: rbtree: fix minor typos in comments
015. V02 | #01 | rust: acpi: replace manual zero-initialization with 'pin_init::zeroed('
016. V02 | #01 | rust: drm: use 'pin_init::zeroed()' for file operations initialization
017. V08 | #06 | rust: add ww_mutex support
018. V01 | #01 | rust: num: bounded: add safety comment for Bounded::__new
019. V05 | #01 | rust: Return Option from page_align and ensure no usize overflow
020. V01 | #01 | rust: id_pool: fix example

Latest Patchsets from rust-for-linux (ESC / q) to return | (ENTER) to select | (h / ←) previous page | (l / →) next page
```

Figure 3: Latest patchsets screen of the rust-for-linux list (redacted patchset authors).

81 **Patchset rendering**

82 For every patchset, users can view its metadata, which includes the title, author, patchset
83 version, and the number of *Reviewed-by*, *Tested-by*, and *Acked-by* tags it has received. Users

84 can also inspect each individual patch within the patchset, as well as the patchset's cover
85 letter. For each patch, the commit message and the *code diff* can be viewed using different
86 renderers for syntax highlighting and colorization. This allows users to track the current review
87 state of each patch and conduct their own review efficiently without the need to set up tools
88 like mutt and do manual integrations with Lore.

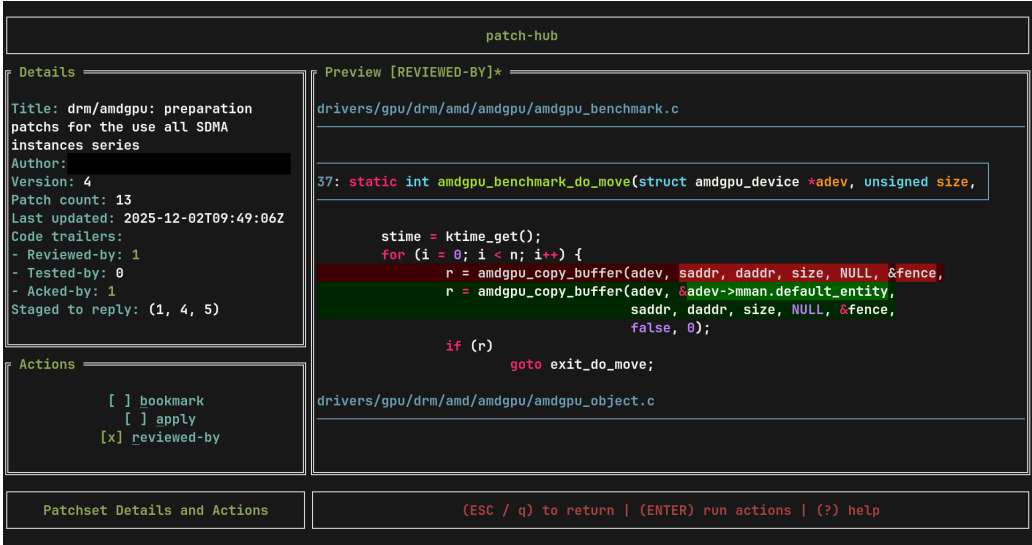


Figure 4: Patchset preview and management screen.

89 **Patchset management**

90 Beyond simply viewing patchsets, users can actively interact with them. Three main actions
91 are supported:

- 92 1. Bookmark a patchset to access it later.
93 2. Apply the patchset to a local kernel tree to validate and test the proposed changes.
94 3. Reply to a patchset (or individual patches in a patchset) with a *Reviewed-by* tag, to
95 indicate endorsement of the contribution.

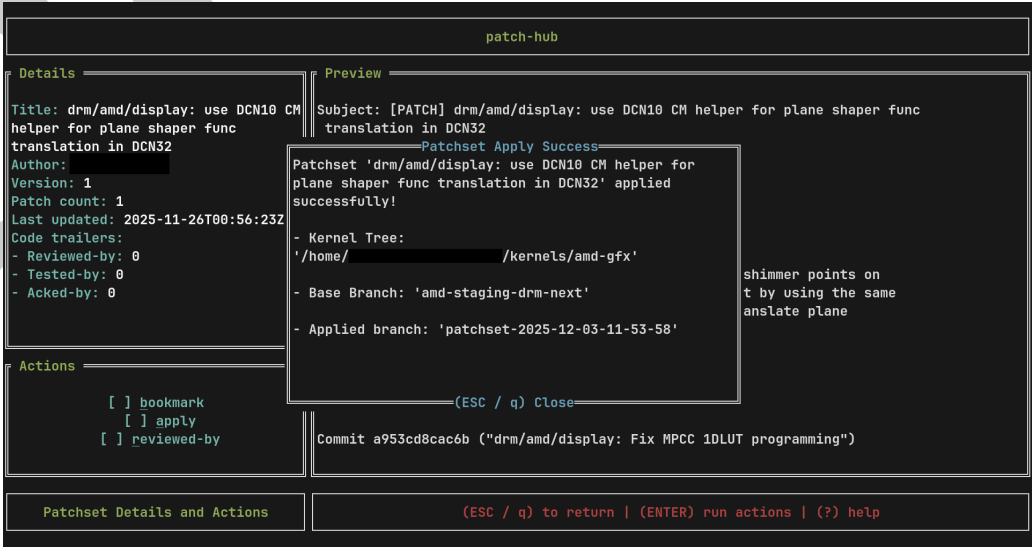


Figure 5: Pop-up after successfully applying patchset.

96 Custom configuration

97 Another important feature is the ability to customize specific system settings. The main
98 options include: selecting which tool will be used to render patchsets, configuring how many
99 patchsets are displayed per page, defining directories for data and cache storage, and setting
100 log retention periods. Users can also configure integration with Git commands, such as git
101 send-email for replying to patchsets and git am for applying a patchset to the local kernel
102 tree. This ensures that the review and application workflow can be tailored to each user's
103 preferences.



Figure 6: Configuration screen

104 Architecture

105 The patch-hub architecture can be divided into two fundamental parts:

- 106 1. The core of the application, which handles all state changes triggered by user interaction.
107 2. The integration with lore.kernel.org, which provides access to up-to-date patchsets.

108 MVC (Model–View–Controller)

109 The core of the application was developed following the *Model–View–Controller* (MVC)
110 design pattern, where the Model represents the aggregation of all application states, the View
111 represents the abstraction of how those states are rendered to the user, and the Controller
112 manages user interactions and requested actions.

113 Model

114 Practically speaking, patch-hub defines a struct named App, which implements the Model
115 layer. In summary, it contains:

- 116 ■ A CurrentScreen attribute, indicating the application's active screen.
117 ■ One struct for each possible screen the user can access (MailingListSelection,
118 BookmarkedPatchsets, LatestPatchsets, DetailsActions, EditConfig).
119 ■ A Config struct that stores the current configuration.
120 ■ A BlockingLoreAPIClient struct that represents the HTTP client responsible for com-
121 municating with lore.kernel.org.

```
pub struct App {
    pub current_screen: CurrentScreen,
    pub mailing_list_selection: MailingListSelection,
    pub bookmarked_patchsets: BookmarkedPatchsets,
    pub latest_patchsets: Option<LatestPatchsets>,
    pub details_actions: Option<DetailsActions>,
    pub edit_config: Option<EditConfig>,
    pub config: Config,
    pub lore_api_client: BlockingLoreAPIClient,

    /// other less relevant attributes omitted
}
```

122 Listing 1: App struct snippet.

123 As expected, the Model layer does not handle either end of the application, user interaction,
124 or terminal rendering, but only the core logic of the system. The App struct is responsible for
125 storing each screen's state, the loaded patchsets, configuration data, and for orchestrating
126 transitions between states. However, it does not directly handle user input or screen rendering.

127 View

128 Since patch-hub is a TUI, the View layer focuses on rendering each screen in the terminal.
129 Concretely, whenever a state change occurs, the terminal is redrawn with the relevant infor-
130 mation for the user, via the `draw_ui()` function. This function retrieves the current screen
131 from the App and renders it according to its definition and current state. To draw widgets,
132 patch-hub utilizes the Rust library Ratatui, which provides definitions for colors, alignments,
133 geometric shapes, and other UI components.

134 Besides rendering individual screens, some UI components, such as loading screens and pop-up
135 windows, can appear across multiple views. Their rendering behavior is also handled within
136 the View layer.

```
pub fn draw_ui(f: &mut Frame, app: &App) {
    /// beggining of the function omitted

    let chunks = Layout::default()
        .direction(Direction::Vertical)
        .constraints([
            Constraint::Length(3),
            Constraint::Min(1),
            Constraint::Length(3),
        ])
        .split(f.area());

    render_title(f, chunks[0]);

    match app.current_screen {
        CurrentScreen::MailingListSelection => mail_list::render_main(f, app, chunks[1])
        CurrentScreen::BookmarkedPatchsets => {
            bookmarked::render_main(f, &app.bookmarked_patchsets, chunks[1])
        }
        CurrentScreen::LatestPatchsets => latest::render_main(f, app, chunks[1]),
        CurrentScreen::PatchsetDetails => details_actions::render_main(f, app, chunks[1]),
        CurrentScreen::EditConfig => edit_config::render_main(f, app, chunks[1]),
    }
}
```

```
navigation_bar::render(f, app, chunks[2]);
```

```
/// rest of the function omitted
```

```
}
```

137 Listing 2: draw_ui() function snippet.

138 Notably, the View layer does not know how information is stored or which user interactions led
139 to the current state. It only needs the current state to decide how to compose and display the
140 interface elements.

141 Controller

142 The Controller layer coordinates the chain of operations triggered by user actions. In general,
143 it captures keyboard events and routes them to their corresponding actions, which typically
144 involve updating the App (Model) and then redrawing the screen (View). Each screen has its
145 own event handler, and whenever a user action causes a screen transition, the corresponding
146 handler function is invoked.

147 Thus, the Controller directly interacts with both the Model and the View, orchestrating their
148 operation at a high level.

```
match key.code {
  KeyCode::Char('?') => {
    let popup = generate_help_popup();
    app.popup = Some(popup);
  }
  KeyCode::Esc | KeyCode::Char('q') => {
    app.reset_latest_patchsets();
    app.set_current_screen(CurrentScreen::MailingListSelection);
  }
  KeyCode::Char('j') | KeyCode::Down => {
    latest_patchsets.select_below_patchset();
  }
  KeyCode::Char('k') | KeyCode::Up => {
    latest_patchsets.select_above_patchset();
  }
}
```

149 Listing 3: Example of Key-to-action routing.

150 Lore API

151 With the MVC structure established, the other main pillar of patch-hub is its integration
152 module with lore.kernel.org, which enables fetching patchsets. This module is not part of
153 the MVC structure because it operates independently of the core business logic; it merely
154 provides data to patch-hub, and could be replaced by another source without requiring changes
155 elsewhere in the system.

156 The primary goal of this module is to communicate with lore.kernel.org via HTTP requests to
157 retrieve the list and details of patchsets.

158 The HTTP client is represented by the BlockingLoreAPIClient struct, which is responsible
159 for managing requests, handling network communication (including headers, timeouts, etc.),
160 and parsing XML responses from the site.

161 Three primary endpoints were implemented to provide all the information patch-hub needs
162 about patches from lore.kernel.org:

- 163 ■ request_available_lists: returns all mailing lists of kernel subsystems available on
164 lore.kernel.org;

- 165 ▪ **request_patch_feed**: given a mailing list, returns the list of patchsets it contains;
- 166 ▪ **request_patch_html**: given a patchset ID, returns the complete information about it
- 167 and its contents.

168 The integration module also includes another struct, `LoreSession`, which acts as an intermediary
 169 between the App screens and the external data source. It is responsible for calling the
 170 `BlockingLoreAPIClient`, processing XML responses, storing loaded patchsets along with their
 171 associated mailing lists, and providing this data to the App whenever required.

172 This design keeps the external data source decoupled from the core application logic, facilitating
 173 both testing and potential future replacement or extension of how patchsets are retrieved.

```
fn request_available_lists(&self, min_index: usize) -> Result<String, ClientError> {
    let available_lists_url = format!("{}/?&o={min_index}", self.lore_domain);

    let body: String = ureq::get(&available_lists_url)
        .header("Accept", "text/html,application/xhtml+xml,application/xml")
        .call()?
        .body_mut()
        .read_to_string()?;
    Ok(body)
}
```

174 Listing 4: Example of HTTP request to Lore.

175 Discussion

176 Rust

177 There are two main motivations behind choosing Rust for the development of patch-hub.
 178 First, although patch-hub does not have strict constraints such as high performance or limited
 179 memory usage, Rust offers several characteristics that provide universal benefits to software
 180 projects. Notably:

- 181 ▪ Memory safety, enforced at compile time, which prevents a wide range of well-known
 182 programming bugs such as use-after-free, dangling pointers, and double free.
- 183 ▪ Idiomatic expressiveness, arising from Rust's language design, which encourages clean
 184 code practices and enhances code readability. Key features include immutability by
 185 default and constructs such as `Result`, `Option`, and functional-style iterator combinators
 186 (`map`, `filter`, `find`, `collect`, etc.).

187 The second motivation is more abstract, directly related to the context in which patch-hub is
 188 situated and reflects recent trends in the Linux kernel development community. The adoption of
 189 Rust in patch-hub aligns with one of the project's implicit goals: modernizing the Linux kernel
 190 development process. In recent years, there has been a significant movement within the kernel
 191 community, even endorsed by Linus Torvalds, toward introducing Rust into this ecosystem.
 192 Although the kernel has historically been written in C, which provides high performance and
 193 a great deal of developer freedom, the motivation for incorporating Rust primarily lies in its
 194 compile-time memory safety guarantees, as mentioned above.

195 Thus, patch-hub follows this growing enthusiasm for the language and aligns itself with the
 196 community's ongoing trends. Moreover, contributing to patch-hub (or to any other open-source
 197 projects written in Rust) can be seen as an opportunity to prepare for future contributions to
 198 the kernel itself, especially considering that one of the key challenges in adopting Rust is its
 199 relatively steep learning curve.

200 Importance

201 A recurring concern among Linux kernel developers in recent years has been the sustainability
202 of the development cycle, particularly given the bottlenecks created by the project's scale and
203 outdated development processes. One possible way to address these challenges, as discussed
204 in (Tadokoro et al., 2025a), is by increasing the use of development support tools, which can
205 help reduce the cognitive and operational burden of tasks that are secondary to the system's
206 evolution.

207 In the context of interacting with patches, users must understand the dynamics of mailing
208 lists and learn the steps and conventions involved in submitting and reviewing patches. These
209 factors can slow down the development cycle and make it harder to integrate new contributors,
210 reviewers, and maintainers.

211 Patch-hub is one such support tool that aims to directly improve this scenario. By allowing users
212 to visualize, validate, and respond to patchsets more quickly, intuitively, and in a centralized
213 manner, the tool eliminates or simplifies many of the steps traditionally required in the process.

214 The lore.kernel.org platform itself is an example of a tool designed to simplify how users interact
215 with patchsets. patch-hub builds on this well-established system, extending its functionality
216 and usability so that users need nothing beyond their terminal to work with patchsets.

217 For these reasons, patch-hub can be viewed as a bridge between the traditional practices of
218 kernel development, which depend on tools and technologies that are increasingly uncommon
219 in modern software engineering, and more contemporary approaches that emphasize user expe-
220 rience as a means to boost productivity and reduce the likelihood of errors. Furthermore, when
221 considered within the broader context of its integration with kw, patch-hub can significantly
222 expand the potential for automation and, consequently, accelerate the entire development
223 workflow.

224 Another point worth highlighting is the tool's potential to serve as a platform for experimentation
225 and metrics collection regarding the kernel contribution process. The analysis of data and
226 feedback generated during its use could enable investigations into different aspects of the
227 patch review cycle — such as review time, volume, and engagement in reviews, among other
228 metrics related to reviewers' interactions with patches.

229 In this way, patch-hub not only facilitates the daily work of contributors but also creates
230 opportunities for comparative studies between its use and the traditional patch review flow,
231 fostering broader discussions about collaboration and efficiency in large-scale projects, and
232 specifically how these factors can impact the future of Linux kernel development.

233 Next steps

234 There are two clear next steps for patch-hub. The first is to improve the tool's integration with
235 its predecessor, kw, so that it becomes possible, for example, to compile and deploy a patch or
236 patchset under review in a more automated way, directly from patch-hub itself. Furthermore,
237 given the strong relationship between the tools, additional initiatives can be undertaken to
238 enhance this integration, making the overall development flow even more centralized and
239 seamless.

240 The second step involves instrumenting patch-hub to enable, with user consent, the collection
241 of telemetry data during its execution. By anonymizing and sending this information to a
242 server, it would be possible to consolidate user data and analyze it to identify bottlenecks,
243 understand usage patterns, and propose improvements that reduce friction in the revision flow.
244 The existence of this metrics collection infrastructure will facilitate the design and comparison
245 of future experiments involving Linux kernel developers. The existence of this metrics collection
246 infrastructure will also facilitate the design and comparison of future experiments involving
247 Linux kernel developers. Finally, it can support broader reflections on tool usage and user
248 behavior, as discussed in the previous section.

249 **Acknowledgements**

250 **References**

- 251 Corbet, J. (2013). *On saying "no"*. Linux Weekly News. <https://lwn.net/Articles/571995/>
- 252 Corbet, J. (2018). *Two perspectives on the maintainer relationship*. Linux Weekly News.
253 <https://lwn.net/Articles/749676/>
- 254 Corbet, J. (2021). *Maintainers truth and fiction*. Linux Weekly News. <https://lwn.net/Articles/842415/>
255
- 256 Dean, S. (2020). *The maintainer's paradox: Balancing project and community*. Linux Founda-
257 tion. <https://www.linuxfoundation.org/blog/the-maintainers-paradox-balancing-project-and-commun>
- 258 Edge, J. (2013). *The kernel maintainer gap*. Linux Weekly News. [https://lwn.net/Articles/](https://lwn.net/Articles/572003/)
259 [572003/](https://lwn.net/Articles/572003/)
- 260 Edge, J. (2016). *On moving on from being a maintainer*. Linux Weekly News. <https://lwn.net/Articles/670088/>
261
- 262 Edge, J. (2018). *Too many lords, not enough stewards*. Linux Weekly News. <https://lwn.net/Articles/745817/>
263
- 264 Rigby, P. C., German, D. M., Cowen, L., & Storey, M.-A. (2014). Peer review on open-source
265 software projects: Parameters, statistical models, and theory. *ACM Trans. Softw. Eng. Methodol.*, 23(4). <https://doi.org/10.1145/2594458>
266
- 267 Tadokoro, D., Siqueira, R., & Meirelles, P. (2025a). Can the linux kernel sustain 30 more years
268 of growth? Toward mitigating bottlenecks in its development model. *Anais Do XXXIX Simpósio Brasileiro de Engenharia de Software*, 845–851. <https://doi.org/10.5753/sbes.2025.11607>
269 [2025.11607](https://doi.org/10.5753/sbes.2025.11607)
270
- 271 Tadokoro, D., Siqueira, R., & Meirelles, P. (2025b). Kworkflow a linux kernel developer
272 automation workflow system. *Anais Do XXXIX Simpósio Brasileiro de Engenharia de Software*, 949–955. <https://doi.org/10.5753/sbes.2025.11322>
273
- 274 Vetter, D. (2017). *Maintainers don't scale*. [https://blog.ffwll.ch/2017/01/maintainers-dont-scale.](https://blog.ffwll.ch/2017/01/maintainers-dont-scale.html)
275 [html](https://blog.ffwll.ch/2017/01/maintainers-dont-scale.html)